TeleCon firmware sketches

Which `.ino` to flash

ESP32 DevKit over Bluetooth, and ESP32-CAM for video

Version 1.0 2026

Abstract

The Android app downloads one ZIP per connection from the firmware repository. Each ZIP is one Arduino sketch, and the sketch is the `.ino` with the same name. This note names every Bluetooth control sketch and the ESP32-CAM sketch. The camera connects only with a DevKit over Bluetooth and a separate ESP32-CAM for video.

DevKit + Bluetooth CAM video only + DevKit

Contents

1 The only way to connect the camera **1**

2 Bluetooth control on an ESP32 DevKit **2**

3 ESP32-CAM, video only **3**

4 What the ZIP contains **3**

1 The only way to connect the camera

The picture does not come from the same board as the sticks. Two boards are required, and the app has no other camera layout.

Role	Board	Sketch
Sticks	ESP32 DevKit	One of the three Bluetooth `.ino` files
Picture	ESP32-CAM	`TeleCon_CAM_-SoftAP_Video.ino`

Flash the video sketch on the CAM and one Bluetooth sketch on the DevKit. On the phone, keep the Bluetooth mode pointed at the DevKit and turn on the camera overlay. The phone joins Wi-Fi `TeleCon-RC-CAM-Starter`, password `telecon1234`, for the picture only. Sticks stay on Bluetooth. One ESP32-CAM does not carry them, and control does not travel on the camera Wi-Fi.

With no camera, the DevKit and one of the three Bluetooth sketches are enough. That board has no video.

2 Bluetooth control on an ESP32 DevKit

Board in the app: **DevKit**. No camera on this board. Open the sketch folder in the Arduino IDE (the folder that contains the `.ino`) and upload to an ESP32 Dev Module.

ZIP / `.ino`	Phone setting	On the air
`TeleCon_Classic_Simpl`	Classic + Simple	Name `ESP32-TC-RC-BT-Simple`, PIN `1234`. Handshake `proto,simple`. Text lines `RC:CTRL`, `RC:BTN`, `RC:DATA`, `RC:PLOT`.
`TeleCon_Classic_Binar`	Classic + Binary	Name `ESP32-TC-RC-BT-Binary`. Handshake `proto,binary`. Frames `AA 55`, `BB 66`, `CC 11/22/33`.
`TeleCon_BLE_Binary`	BLE Binary	Name `ESP32-TC-RC-BLE-Binary` (Nordic UART). Same binary frames as Classic Binary.

`TeleCon_Classic_Simple.ino` Single file for a first board. Put motor and servo code in `onControl()` and `onButton()` where the file says `YOUR CODE HERE`. A steering servo on GPIO 13 is already wired in the sketch: stick `rx` = 0 is a 1500 µs pulse. Eight telemetry bytes (phone CH1–CH8) are chosen in `kTelemetryChannels[]` in this same file. With `ENABLE_PLOT` at 1 the board sends `RC:PLOT` about every 50 ms. The code guide for this sketch is `telecon_classic_simple_code.tex` in this documentation set.

`TeleCon_Classic_Binary.ino` The `.ino` only starts Bluetooth and the vehicle modules. Pins, PWM, and the eight telemetry channels live in the other files in that folder. Edit `pins.h` for your driver and `TelemetryChannels.h` for what CH1–CH8 report. The phone setting must be Classic + Binary before connect. The wire format is the control protocol document, not the text lines of the Simple sketch.

`TeleCon_BLE_Binary.ino` Same vehicle modules and the same `TelemetryChannels.h` table as Classic Binary. The link is BLE instead of Classic SPP. Choose BLE Binary in the app. Do not pair it as a Classic device.

These three sketches are also the control half of the two-board camera layout below. Flash only one of them on the DevKit.

3 ESP32-CAM, video only

Use this when a DevKit already runs one of the Bluetooth sketches and you want a picture from a second board.

`TeleCon_CAM_SoftAP_Video.ino` ZIP `TeleCon_CAM_SoftAP_Video.zip`. Flash an ESP32-CAM (AI-Thinker). This sketch serves video and does not accept sticks.

- SoftAP `TeleCon-RC-CAM-Starter`, password `telecon1234`, address `192.168.4.1`.
- Picture: `http://192.168.4.1/stream` (MJPEG) or `/capture` (one JPEG).
- Stream quality: `http://192.168.4.1/camconfig`.
- No control port. There is no `RC:CONNECT` on this board.

On the phone, turn on the camera overlay and keep the Bluetooth connection pointed at the DevKit (Classic Simple, Classic Binary, or BLE Binary). The phone uses Wi-Fi for the CAM and Bluetooth for the sticks at the same time. Do not put stick control on the ESP32-CAM.

4 What the ZIP contains

The app downloads `zips/TeleCon_<name>.zip`. Unzip it and open the folder that contains the `.ino` of the same name. Arduino compiles every `.cpp` and `.h` in that folder, so a binary sketch is the `.ino` plus those modules. Do not copy `SteerCenter` back in: stock sketches do not store a steering center. Trim stays on the phone, and `rx` $= 0$ is 1500 µs.